

Finite Element Methods – APE4

Matrix assembly, geometry and element mapping

1 Modifying the code: matrix assembly and mesh generation

We are interested in solving the Poisson problem defined on a 2D domain Ω :

$$\begin{cases} -\Delta u = f & \text{in } \Omega, \\ u = g & \text{on } \partial\Omega. \end{cases}$$

The weak formulation of this problem is: find $u \in H^1(\Omega)$ such that $u = g$ on $\partial\Omega$ and

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in H_0^1(\Omega).$$

In the finite element method, we approximate u by a function u_h in a finite-dimensional subspace $V_h \subset H^1(\Omega)$, typically consisting of piecewise polynomial functions defined on a mesh of Ω . The discrete problem is: find $u_h \in V_h$ such that $u_h = g$ on $\partial\Omega$ and

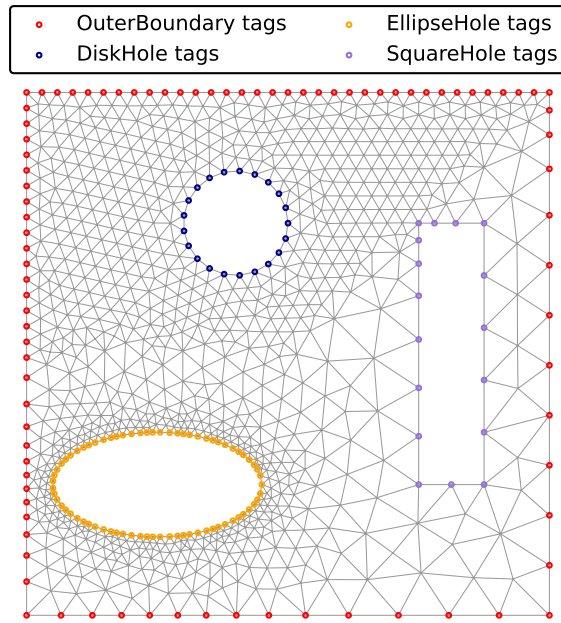
$$\int_{\Omega} \nabla u_h \cdot \nabla v_h \, dx = \int_{\Omega} f v_h \, dx \quad \forall v_h \in V_h^0,$$

where $V_h^0 = \{v_h \in V_h : v_h = 0 \text{ on } \partial\Omega\}$. To solve this problem, we need to assemble the stiffness matrix A and the load vector b corresponding to the bilinear form $a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx$ and the linear form $L(v) = \int_{\Omega} f v \, dx$. We can compute the entries of A and b such that

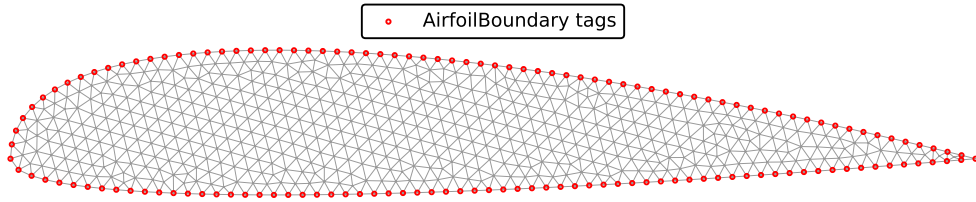
$$K_{ij} = a(\phi_i, \phi_j) = \int_{\Omega} \nabla \phi_i \cdot \nabla \phi_j \, dx, \quad F_i = \ell(\phi_i) = \int_{\Omega} f \phi_i \, dx,$$

where $\{\phi_i\}$ is the classical "tent" basis function of V_h associated to the node i . A naive way to compute these integrals is to loop over all pairs of basis functions ϕ_i and ϕ_j , compute the integrals, and fill the matrix A and vector b . However, this approach is inefficient because most pairs of basis functions have disjoint support, leading to many zero entries in A . Instead, we can use the fact that the basis functions are defined locally on each element of the mesh. We can loop over each element, compute the local stiffness matrix and load vector for that element, and then assemble these local contributions into the global matrix and vector. This process is known as *matrix assembly*. This is what you learned in the lecture with Prof. Dr. Ir. Legat :- (almost arch, sad), and it is the standard approach in finite element implementations.

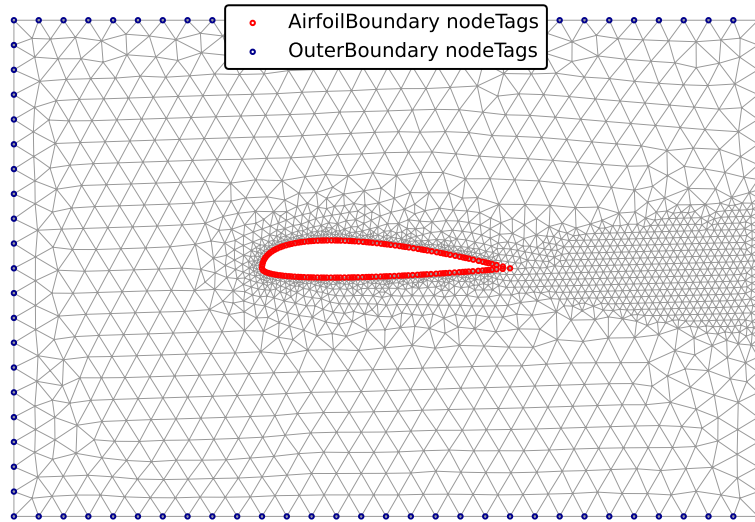
1. In the code provided, the matrix assembly is performed in this inefficient naive way, by looping over all pairs of basis functions. This is done in the function `assemble_stiffness_and_rhs_node_wise`. Modify this function to implement the more efficient element-wise assembly approach. You will need to loop over each element, compute the local stiffness matrix and load vector for that element using quadrature, and then add these local contributions to the global matrix K and vector F .
2. The code use a mesh generated by Gmsh, which consists of triangular elements. Two example meshes are provided and shown in the figure 1a and 1b. Combine those examples to create a new mesh that consists of a rectangular domain with a airfoil-shaped cut embedded and varying mesh sizes, as shown in the figure 1c.



(a) Mesh generated from `mesh1(L=1, H=1, order=1, smin=0.01, smax=0.1, dmin=0, dmax=0.3)`



(b) Mesh generated from `mesh2(L=1, H=1, order=1, smin=0.01, smax=0.015, dmin=0, dmax=0.3)`



(c) TODO : A rectangular mesh, cut with the airfoil shape. The SizeField (meshSize) is defined by a minimum of a field `Distance` and a field `MathEval`.

Figure 1: Mesh generation using the GMSH API

2 Element mapping

Consider a reference triangle $\hat{T}$ with vertices at $(0, 0)$, $(1, 0)$, and $(0, 1)$. Let T be a triangle in the physical domain with vertices at (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) . We will consider the following cases:

- T is the triangle with vertices at $(0, 0)$, $(2, 0)$, and $(0, 2)$.
- T is the triangle with vertices at $(1, 1)$, $(2, 1.5)$, and $(1.5, 2)$.

For each of these cases, we have an affine mapping going from the physical triangle T to the reference triangle $\hat{T}$, defined by $x(\xi, \eta)$ and $y(\xi, \eta)$. The Jacobian matrix of this mapping is given by

$$J = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{bmatrix},$$

and its determinant is given by $\det(J) = \frac{\partial x}{\partial \xi} \frac{\partial y}{\partial \eta} - \frac{\partial x}{\partial \eta} \frac{\partial y}{\partial \xi}$.

1. For each physical triangle T , compute the Jacobian matrix J and its determinant $\det(J)$ for the mapping from the reference triangle $\hat{T}$ to the physical triangle T .
2. Check your results with the code provided, by creating a one element mesh and printing the jacobian matrix and its determinant for each of these cases.
3. Compute by hand the gradients of the basis functions $\nabla \phi_i$ in the physical triangle T (either by using the chain rule and the Jacobian matrix, or by using geometry).
4. Check your results with the code provided, by creating a one element mesh and printing the gradients of the basis functions in the physical triangle T for each of these cases.